

Data Engineering Take-Home Assessment

E-Commerce Pipeline Challenge

Duration 4 – 6 hours	Stack PySpark 3.x · Python 3.9+	Submission GitHub repo link	Level
--------------------------------	---	---------------------------------------	-------

1. Background

Welcome, and thank you for taking the time to complete this assessment. You have been given a dataset that mirrors the kind of raw, imperfect data our team works with every day. Your challenge is to build a PySpark ETL pipeline that takes four messy CSV files from ingestion through to a clean, enriched analytical layer — then answer business questions from the results.

This task is designed to take 4–6 hours. We are not looking for perfection — we are looking for sound engineering judgment, clean code, and clear thinking. If you hit a genuine blocker, document it and move on.

2. Your Dataset

Use the provided `generate_data.py` script (or `generate_data_notebook.ipynb` for Jupyter) to produce your four CSV files. Run it once and work from the output it generates.

Run the data generator

```
pip install faker pandas numpy
python generate_data.py          # creates ./data/ with all four CSVs
```

Options: `--rows 2000 --seed 42 --out ./data`

Dataset tables

You will work with four CSV files that together represent a simplified e-commerce data model:

- `orders.csv` — one row per order with customer reference, date, status, amount, and discount.
- `order_items.csv` — line items per order, including product, quantity, unit price, and category.
- `customers.csv` — customer master data including tier, country, and signup date.
- `returns.csv` — return events referencing orders, with reason and refund amount.

Intentional data quality issues

The data contains the following issues. Your pipeline must handle every one of them explicitly — not silently:

- **Duplicates:** ~8% exact duplicate rows across all four tables
- **Date formats:** Mix of YYYY-MM-DD and DD/MM/YYYY across date columns
- **NULL key fields:** ~4% of orders have a NULL customer_id; ~2% have a NULL total_amount
- **Negative amounts:** ~3% of orders have a negative total_amount (flag, do not drop)
- **Orphaned items:** ~4% of order_items reference order IDs that do not exist in orders.csv
- **Casing:** customer_tier values use inconsistent casing (bronze, Bronze, BRONZE, etc.)
- **Refund anomalies:** ~5% of returns have a refund_amount exceeding the original order total

3. Tasks

Complete all six tasks in order. Each task builds on the output of the previous one.

#	Task	Key skills assessed
01	Data ingestion & schema enforcement	SparkSession, StructType, explicit casting
02	Data quality & cleaning	dropDuplicates, date normalisation, NULL handling, flagging
03	Joins & enrichment	Inner / left / anti joins; net_amount derivation
04	Aggregations & window functions	Window, partitionBy, rank, rolling counts
05	Return analysis	Return rate, top refund customers, anomaly flagging
06	Output & partitioning	Parquet write, partitionBy year/month, idempotent overwrite

Task 01 — Data ingestion & schema enforcement

Load all four CSVs with explicit schemas defined in code. Do not use inferSchema=True. Cast each column to its correct data type. Rows that cannot be cast must be collected into a separate 'rejected' DataFrame and logged — not silently dropped.

Task 02 — Data quality & cleaning

Apply the following cleaning steps in sequence:

1. Remove exact duplicate rows using dropDuplicates().
2. Normalise all date columns to ISO format (YYYY-MM-DD).
3. Standardise customer_tier to lowercase.
4. Drop rows where order_id or customer_id is NULL.
5. Add a boolean column is_negative_amount to flag (not drop) orders with total_amount < 0.

Task 03 — Joins & enrichment

Join orders to customers and order_items using the appropriate join type for each relationship. Use an anti-join to isolate orphaned order_items and write them to a separate output file. Add a derived column $\text{net_amount} = \text{total_amount} \times (1 - \text{discount_pct} / 100)$.

Task 04 — Aggregations & window functions

Using PySpark window functions (not `groupBy` alone), produce the following outputs:

6. Customers ranked by lifetime net spend within each country.
7. A 7-day rolling order count per customer (based on `order_date`).
8. Each product category's share of total revenue per calendar month.

Task 05 — Return analysis

Join the returns table to your enriched orders. Compute the return rate (`returns / orders`) per category and per `customer_tier`. Identify the top 10 customers by total refund amount. Add a boolean column `refund_exceeds_order` to flag returns where `refund_amount > net_amount`.

Task 06 — Output & partitioning

Write your final enriched dataset to Parquet, partitioned by order year and order month. Write aggregated summary tables to CSV. All writes must use `mode('overwrite')` so the pipeline is idempotent.

4. Bonus Challenges

These are optional. Completing them will differentiate your submission, but attempting the core six tasks well is more important than rushing to complete all bonuses.

- **B1 — Unit tests:** Write at least 3 pytest unit tests for your cleaning or transformation functions using a small synthetic DataFrame.
- **B2 — Broadcast join:** Identify which join in your pipeline benefits from a broadcast hint, apply it, and explain why in a code comment.
- **B3 — DQ gate:** Add a data quality gate before the final write. Assert no order has a NULL customer after joining, and that `net_amount ≥ 0`. Raise an exception if checks fail.
- **B4 — Explain plan:** Call `df.explain(mode='formatted')` on your most complex transformation and include a brief annotation in your README.

5. Deliverables

Submit a link to a public (or shared) GitHub repository containing the following:

9. README.md — setup instructions, how to run, assumptions, known limitations, and any design decisions worth explaining.
10. pipeline.py (or an equivalent modular structure) — the full PySpark ETL, runnable from a single entry point.
11. A sample of output files committed to the repo, or clear instructions in the README to reproduce them.
12. (Bonus) tests/ directory with pytest files if you completed B1.
13. requirements.txt or pyproject.toml with all dependencies pinned.

6. Rules & Guidance

Environment

We will run your pipeline locally with PySpark 3.4 on Python 3.10. Your SparkSession must be created inside the script — do not assume a running cluster or Databricks environment.

What we want to see

- Native Spark functions (F.to_date, F.regexp_replace, F.when, etc.) over Python UDFs for simple operations.
- Correct join type selection — inner vs left vs anti. Justify non-obvious choices with a comment.
- No .collect() or .toPandas() inside the pipeline logic (only acceptable in tests or final display).
- Functions with a single, clear responsibility rather than one monolithic script.

What we do not penalise

- Not completing every bonus task.
- Using a slightly different project structure than suggested, as long as the README explains it.
- Asking a clarifying question via email if you are genuinely blocked on an ambiguity in the spec.

A note on timing

We trust you to respect the 4–6 hour guideline. If you find yourself going significantly over, stop, commit what you have, and document what you would have done next in your README. A clear write-up of your reasoning is worth more than a rushed Task 06.

7. Evaluation Rubric

Your submission will be scored as follows. Total available: 95 points + up to 20 bonus points.

Area	What we look at	Points
Correctness	Transformations produce accurate results. Joins don't inflate or deflate row counts. Aggregations match expectations.	30
Data quality handling	All stated quality issues addressed explicitly. Edge cases handled without silent data loss.	20
PySpark idioms	Native Spark functions used (not Python UDFs for simple ops). Correct window function usage. No unnecessary collect() or toPandas().	20
Code quality	Modular functions, clear naming, minimal repetition, commented where non-obvious, runnable from a single entry point.	15
README & write-up	Setup instructions that work. Brief explanation of design decisions, assumptions, and limitations.	10
Bonus challenges	Quality of any completed bonus tasks (unit tests, broadcast hint, DQ gates, explain plan).	+5 each

8. Submission

When you are ready to submit, send your GitHub repository link by email to your recruiting contact with the subject line:

[Data Engineering] Take-Home Submission · [Your Full Name]

Please include in the email body:

- A one-sentence summary of the approach you took (just enough for us to know where to start reading).
- Any parts you were unable to complete and why.
- Approximate time spent.

Good luck!

We genuinely enjoy reading these submissions. Show us how you think, not just what you know.